AKTU

B.Tech 5th Sem

GATEWAY CLASSES

CS IT & CS Allied

Object Oriented System Design with C++ (BCS054)

GATEWAY CLASSES

Notes

as per AKTU Syllabus

Unit-5

By Amol Sharma sir

- Pursuing Ph.D. from IIT (BHU)
- Wipro Certified Faculty (WCF)

Objects and Classes

CLASS:

Class is a template for set of objects that share common properties and relationships .In C++, a class is a new data type that contains member variables and member functions that operates on the variables. A class is defined with the keyword class. It allows the data to be hidden, if necessary from external use. When we are defining a class, we are creating a new abstract data type that can be treated like any other built in data type.

Generally a class specification has two parts:-
a) Class declaration
b) Class function definition

the class declaration describes the type and scope of its members. The class function definition describes how the class functions are implemented.

Syntax:-

```
class class-name
{
private:
variable declarations;
function declaration ;
public:
variable declarations;
function declaration;
};
```

The members that have been declared as private can be accessed only from within the class. On the other hand, public members can be accessed from outside the class also. The data hiding is the key feature of oops. The use of keywords private is optional. By default; the members of a class are private.

The variables declared inside the class are known as data members and the functions are known as members functions. Only the member functions can have access to the private data members and private functions. However, the public members can be accessed from the outside the class. ***The binding of data and functions together into a single class type variable is referred to as encapsulation.***

Syntax:-

```
class item
{
        int member;
        float cost;
        public:
                void getldata (int a ,float b);
                void putdata (void);
};
```

GATEWAY CLASSES

The class item contains two data members and two function members, the data members are private by default while both the functions are public by declaration. The function getdata() can be used to assign values to the member variables member and cost, and putdata() for displaying their values . These functions provide the only access to the data members from outside the class.

OBJECTS:

Once a class has been declared we can create variables of that type by using the class name.

Example:

item x;

creates a variables x of type item. In C++, the class variables are known as objects.
Therefore x is called an object of type item.

item x, y ,z also possible.

```
class item
{
-----------
-----------
-----------
}x ,y ,z;
```

would create the objects x ,y ,z of type item.

ACCESSING CLASS MEMBER:

The private data of a class can be accessed only through the member functions of that class. The main() cannot contains statements that the access number and cost directly.

Syntax:
object name.function-name(actual arguments);

Example:- x. getdata(100,75.5);
It assigns value 100 to number, and 75.5 to cost of the object x by implementing the getdata() function .

similarly the statement
x. putdata ( ); //would display the values of data members.

x. number = 100 is illegal. Although x is an object of the type item to which number belongs , the number can be accessed only through a member function and not by the object directly.

GATEWAY CLASSES

DEFINING MEMBER FUNCTION:

Member can be defined in two places:

- Outside the class definition
- Inside the class function

1. **OUTSIDE THE CLASS DEfINAT1ON:**

Member function that are declared inside a class have to be defined separately outside the class.Their definition are very much like the normal functions.

An important difference between a member function and a normal function is that a member function incorporates a membership. Identify label in the header. The 'label' tells the compiler which class the function belongs to.

Syntax:

```
return type class-name::function-name(argument declaration )
{
function-body
}
```

The member ship label class-name :: tells the compiler that the function function -name belongs to the class class-name . That is the scope of the function is restricted to the class-name specified in the header line. The :: symbol is called ***scope resolution operator***.

Example:

```
void item :: getdata (int a , float b )
{
        number=a;
        cost=b;
}
void item :: putdata ( void)
{
        cout<<"number=:"<<number<<endl; cout<<"cost="<<cost<<endl;
}
```

The member functions have some special characteristics that are often used in the program development.

• Several different classes can use the same function name. The "membership label" will resolve their scope, member functions can access the private data of the class .A non member function can't do so.

• A member function can call another member function directly, without using the dot operator.

2. **INSIDE THE CLASS DEF1NATION:**

Another method of defining a member function is to replace the function declaration by the actual function definition inside the class. When a function is defined inside a class, it is treated as an inline function. Normally, only small functions are defined inside the class definition.

Example:

```
class item
```

GATEWAY CLASSES

```
{
        int number;
        float cost;

        public:
                void getdata (int a ,float b);
                void putdata(void)
                {
                        cout<<number<<endl; cout<<cost<<endl;
                }
};
```

Example Program on Classes and Objects:

```
# include<iostream>
using namespace std;
class Item
{
        int number;
        float cost;
        public:
        void getdata( int a , float b);
        void putdata( void)
        {
                cout<<"Number:"<<number<<endl;
                cout<<"Cost :"<<cost<<endl;
        }
};
void Item :: getdata (int a , float b)
{
        number=a;
        cost=b;
}
int main()
{
        Item x;
        cout<<"\n Object x:"<<endl;
        x.getdata( 100,299.95);
        x.putdata();
        Item y;
        cout<<"\n Object y:"<<endl;
        y.getdata(200,175.5);
        y.putdata();
}
```

GATEWAY CLASSES

STATIC DATA MEMBERS AND MEMBER FUNCTIONS:

- **STATIC DATA MEMBER:**
  A data member of a class can be qualified as static .

The properties of a static member variable are similar to that of a static variable. A static member variable has special characteristics.

1) It is initialized to zero when the first object of its class is created. No other initialization is permitted.

2) Only one copy of that member is created for the entire class and is shared by all the objects of that class, no matter how many objects are created.

3) It is visible only with in the class but its life time is the entire program. Static variables are normally used to maintain values common to the entire class. For example a static data member can be used as a counter that records the occurrence of all the objects.

int item :: count; // definition of static data member

Note that the type and scope of each static member variable must be defined outside the class definition .This is necessary because the static data members are stored separately rather than as a part of an object.

Example :-

```
# include<iostream>
using namespace std;

class Item
{
        static int count; //count is static
        int number;
        public:
                void getdata(int a)
                {
                        number=a;
                        count++;
                }
                void getcount(void)
                {
                        cout<<"count:";
                        cout<<count<<endl;
                }
};

int Item :: count ; //count defined

int main( )
{
        Item a,b,c;
```

GATEWAY CLASSES

```
        a.getcount( );
        b.getcount( );
        c.getcount( );

        a.getdata(10);
        b.getdata(20);
        c.getdata(30);

        cout<<"After reading data : "<<endl;
        a.getcount( );
        b.getcount( );
        c.getcount( );
        return(0);
}
```

The output would be

```
count:0
count:0
count:0
After reading data :
count:3
count:3
count:3
```

The static variable count is initialized to Zero when the objects are created. The count is incremented whenever the data is read into an object. Since the data is read into objects three times the variable count is incremented three times. Because there is only one copy of count shared by all the three objects, all the three output statements cause the value 3 to be displayed.

- **<u>STATIC MEMBER FUNCTIONS:</u>**
  A member function that is declared static has following properties:-

1. A static function can have access to only other static members declared in the same class.
2. A static member function can be called using the class name as follows:-

class - name :: function - name;

Example:-

```
#include<iostream>
using namespace std;

class Test
{
        int code;
        static int count; // static member variable
        public:
                void set(void)
```

GATEWAY CLASSES

```
        {
                code=++count;
        }
        void showcode(void)
        {
                cout<<"object member : "<<code<<endl;
        }
        static void showcount(void)
        {
                cout<<"count="<<count<<endl;
        }
};
int Test::count;
int main()
{
        Test t1,t2;
        t1.set( );
        t2.set( );
        Test :: showcount ( );

        Test t3;
        t3.set( );
        Test:: showcount( );

        t1.showcode( ) ;
        t2.showcode( );
        t3.showcode( );

        return 0;
}
```

Output:

```
count=2
count=3
object member : 1
object member : 2
object member : 3
```

GATEWAY CLASSES

FRIEND FUNCTIONS:

We know private members cannot be accessed from outside the class. That is a non - member function can't have an access to the private data of a class. However there could be a case where two classes manager and scientist, have been defined we should like to use a function income-tax to operate on the objects of both these classes.
In such situations, C++ allows the common function to be made friendly with both the classes , there by following the function to have access to the private data of these classes .Such a function need not be a member of any of these classes.
To make an outside function "friendly" to a class, we have to simply declare this function as a friend of the classes as shown below:

```
class ABC
{
---------
---------
public:
--------
----------
friend void xyz(void);
};
```

The function declaration should be preceded by the keyword friend, the function is defined elsewhere in the program like a normal C ++ function. The function definition does not use the keyword friend or the scope operator **::** . The functions that are declared with the keyword friend are known as friend functions. A function can be declared as a friend in any no. of classes. Friends function, as though not a member function has full access rights to the private members of the class.

- A friend function processes certain special characteristics:
- It is not in the scope of the class to which it has been declared as friend.
- Since it is not in the scope of the class, it cannot be called using the object of that class. It can be invoked like a member function without the help of any object.
- Unlike member functions.

GATEWAY CLASSES

Example:

```
#include<iostream>
using namespace std;

class Sample
{
        int a;
        int b;
        public:
                void setvalue( )
                {
                        a=25;
                        b=40;
```

```
        }
        friend float mean( Sample s);
};
float mean (Sample s)
{
    return (float(s.a+s.b)/2.0);
}
int main()
{
    Sample x;
    x.setvalue( );
    cout<<"mean value="<<mean(x)<<endl;
    return 0;
}
```

Output:

mean value=32.5

<u>A function friendly to two classes:</u>

```
#include<iostream>
using namespace std;

class abc;
class xyz
{
    int x;
    public:
        void setvalue(int i) { x=i; }
        friend void max (xyz,abc);
};
class abc
{
    int a;
    public:
        void setvalue(int i) {a=i; }
        friend void max(xyz,abc);
};
void max( xyz m, abc n)
{
    if(m.x>=n.a)
        cout<<m.x;
    else
        cout<<n.a;
}
int main( )
{
    abc j;
    j.setvalue(10);
    xyz s;
```

GATEWAY CLASSES

```
        s.setvalue(20);
        max( s , j );
        return 0;
}
```

SWAPPING PRIVATE DATA OF CLASSES:

```
#include<iostream>
using namespace std;

class class2;
class class1
{
        int value1;
        public:
                void indata( int a) { value1=a; }
                void display(void) { cout<<value1<<endl; }
                friend void exchange ( class1 &, class2 &);
};

class class2
{
        int value2;
        public:
                void indata( int a) { value2=a; }
                void display(void) { cout<<value2<<endl; }
                friend void exchange(class1 & , class2 &);
};

void exchange( class1 &x, class2 &y)
{
        int temp=x.value1;
        x.value1=y.value2;
        y.value2=temp;
}
int main()
{
        class1 c1;
        class2 c2;
        c1.indata(100);
        c2.indata(200);
        cout<<"values before exchange:"<<endl;
        c1.display();
        c2.display();
        exchange (c1,c2);
        cout<<"values after exchange :"<< endl;
        c1.display();
        c2.display();
```

GATEWAY CLASSES

```
	return(0);
}
```

Output:
values before exchange
100
200
values after exchange
200
100

<u>PROGRAM FOR ILLUSTRATING THE REAL TIME USE OF FRIEND FUNCTION:</u>

```
#include<iostream>
using namespace std;

class account1;
class account2
{
	private:
		int balance;
	public:
		account2() { balance=567; }
		void showacc2()
		{
			cout<<"balanceinaccount2 is:"<<balance<<endl;
		}
		friend int transfer (account2 &acc2, account1 &acc1,int amount);
};
class account1
{
	private:
		int balance;
	public:
		account1() { balance=345; }
		void showacc1 ()
		{
			cout<<"balance in account1 :"<<balance<<endl;
		}
		friend int transfer (account2 &acc2, account1 &acc1 ,int amount);
};
int transfer ( account2 &acc2, account1 & acc1, int amount)
{
	if(amount <=acc1.balance)
	{
		acc2. balance += amount;
		acc1 .balance -= amount;
		return amount;
	}
```

GATEWAY CLASSES

```
        else
                return(0);
}
int main()
{
        account1 aa;
        account2 bb;
        cout << "balance in the accounts before transfer:" ;
        aa.showacc1();
        bb.showacc2();
        cout << "amt transferred from account1 to account2 is:";
        cout<<transfer( bb,aa,100)<<endl;
        cout<< " balance in the accounts after the transfer:";
        aa.showacc1();
        bb.showacc2();
        return 0;
}
```

Output:

```
balance in the accounts before transfer:balance in account1 :345
balanceinaccount2 is:567
amt transferred from account1 to account2 is:100
 balance in the accounts after the transfer:balance in account1 :245
balanceinaccount2 is:667
```

GATEWAY CLASSES

CONSTRUCTOR:

A constructor is a special member function whose task is to initialize the objects of its class. It is special because its name is the same as the class name. The constructor is invoked whenever an object of its associated class is created. It is called constructor because it construct the values of data members of the class.

A constructor is declared and defined as follows:

```
//'class with a constructor
class integer
{
        int m,n:
        public:
        integer(void);//constructor declared
        ------------
        ------------
};
integer :: integer(void)
{
        m=0;
        n=0;
}
```

When a class contains a constructor like the one defined above it is guaranteed that an object created by the class will be initialized automatically.

For example:-
Integer int1; //object int1 created
This declaration not only creates the object int1 of type integer but also initializes its data members, m and n to zero.

The constructor functions have some characteristics:-

1. They should be declared in the public section.
2. They are invoked automatically when the objects are created.
3. They don't have return types, not even void and therefore they cannot return values.
4. They cannot be inherited, though a derived class can call the base class constructor.
5. Like other C++ function, they can have default arguments,
6. Constructor can't be virtual.

DEFAULT CONSTRUCTOR:

A constructor that accepts no parameter is called the ***default constructor***. The default constructor for class A is A :: A( ). If no such constructor is defined, then the compiler supplies a default constructor.

Therefore a statement such as:-
A a ; //invokes the default constructor of the compiler of the compiler to create the object "a" ;
Invokes the default constructor of the compiler to create the object a.

GATEWAY CLASSES

Example of default constructor:

```
#include<iostream>
using namespace std;

class abc
{
	private:
		char nm[];
	public:
		abc()
		{
			cout<<"enter your name:";
			cin>>nm;
		}
		void display()
		{
			cout<<nm;
		}
};
int main()
{
	abc d; //Default constructor will be invoked here
	d.display();

	return 0;
}
```

PARAMETERIZED CONSTRUCTOR:

The constructors that can take arguments are called parameterized constructors. Using parameterized constructor we can initialize the various data elements of different objects with different values when they are created.

Example:

```
class integer
{
	int m,n;
	public:
	integer( int x, int y);
	--------
	---------
};

integer:: integer (int x, int y)
{
	m=x;n=y;
}
```

GATEWAY CLASSES

the argument can be passed to the constructor by calling the constructor implicitly or explicitly.

integer int 1 = integer(0,100); // explicit call
integer int 1(0,100); //implicit call

Example of parameterized constructor:

```
#include<iostream>
using namespace std;

class integer
{
	int m,n;
	public:
		integer(int,int);
		void display(void)
		{
			cout<<"m= "<<m<<endl ;
			cout<<"n= "<<n<<endl;
		}
};
integer :: integer(int x,int y) // constructor defined
{
	m=x;
	n=y;
}

int main( )
{
	integer int1(0, 100); // implicit call
	integer int2=integer(25,75);
	cout<<"\nObjectl "<<endl;
	int1.display( );
	cout<<"\nObject2 "<<endl;
	int2.display( );
	return 0;
}
```

<u>COPY CONSTRUCTOR:</u>

A copy constructor is used to declare and initialize an object from another object.
Example:-
the statement

integer i2(i1);

would define the object 12 and at the same time initialize it to the values of i1.
Another form of this statement is:

integer i2=i1;

The process of initialization through a copy constructor is known as copy initialization.

GATEWAY CLASSES

Example of copy constructor:

```
#incliide<iostream.h>
#include<iostream>
using namespace std;

class code
{
	int id;
	public:
		code() { } //constructor
		code (int a) { id=a; } //constructor
		code(code &x)
		{
			id=x.id;
		}
		void display()
		{
			cout<<id;
		}
};
int main()
{
	code A(100);
	code B(A);
	code C=A;
	code D;
	D=A;
	cout<<" \n id of A :";
	A.display();
	cout<<" \nid of B :";
	B.display();
	cout<<" \n id of C:";
	C.display();
	cout<<" \n id of D:";
	D.display();

	return 0;
}
```

GATEWAY CLASSES

<u>CONSTRUCTOR OVERLOADING:</u>

In C++, We can have more than one constructor in a class with same name, as long as each has a different list of arguments.This concept is known as Constructor Overloading and is quite similar to function overloading.

- Overloaded constructors essentially have the same name (name of the class) and different number of arguments.
- A constructor is called depending upon the number and type of arguments passed.

- While creating the object, arguments must be passed to let compiler know, which constructor needs to be called.

Example:

```
#include<iostream>
using namespace std;

class sum
{
    int a;
    int b;

    public:
            sum();
            sum(int a);
            sum(int a,int b);

            void display();
};
sum :: sum()
{
    a=0;
    b=0;
}
sum :: sum(int x)
{
    a=x;
    b=x;
}
sum :: sum(int x,int y)
{
    a=x;
    b=y;
}

void sum :: display()
{
    cout<<"a= "<<a<<endl;
    cout<<"b= "<<b<<endl;
}

int main()
{
    sum l;
    l.display();
    sum m(20);
    m.display();
```

GATEWAY CLASSES

```
    sum n(30,40);
    n.display();

    return 0;
}
```

DESTRUCTOR:

A destructor, as the name implies is used to destroy the objects that have been created by a constructor. Like a constructor, the destructor is a member function whose name is the same as the class name but is preceded by a tilde.
For Example:-

~ integer( ) { }

A destructor never takes any argument nor does it return any value. It will be invoked implicitly by the compiler upon exit from the program to clean up storage that is no longer accessible. It is a good practice to declare destructor in a program since it releases memory space for future use.

Delete is used to free memory which is created by new.

```
matrix : : ~ matrix( )
{
for(int i=0; i<11;i++)
delete p[i];
delete p;
}
```

Example:

```
#include <iostream>

using namespace std;

int count=0;
class alpha
{
   public:
      alpha()
      {
         count ++;
         cout<<"No of object created: "<<count<<endl;
      }
      ~alpha()
      {
         cout<<"No of object destroyed: "<<count<<endl;
         count--;
      }
};
int main()
{
```

GATEWAY CLASSES

```
    cout<<"Enter main \n:";
    alpha A1,A2,A3,A4;
    {
        cout<<"Enter block 1 :\n";
        alpha A5;
    }
    {
        cout<<"Enter block2 \n";
        alpha A6;
    }
    cout<<"Re-enter main \n:";
    return 0;
}
```

Output:

```
Enter main
:No of object created: 1
No of object created: 2
No of object created: 3
No of object created: 4
Enter block 1 :
No of object created: 5
No of object destroyed: 5
Enter block2
No of object created: 5
No of object destroyed: 5
Re-enter main
:No of object destroyed: 4
No of object destroyed: 3
No of object destroyed: 2
No of object destroyed: 1
```

GATEWAY CLASSES

OPERATOR OVERLOADING

Operator overloading provides a flexible option for the creation of new definitions for most of the C++ operators.

Although the semantics of an operator can be extended, we can't change its syntax, the grammatical rules that govern its use such as the no of operands precedence and associativity. For example the multiplication operator will enjoy higher precedence than the addition operator.

When an operator is overloaded, its original meaning is not lost. For example, the operator +, which has been overloaded to add two vectors, can still be used to add two integers.

To define an additional task to an operator, we must specify what it means in relation to the class to which the operator is applied. This is done with the help of a special function called operator function, which describes the task.

Syntax:-

```
return-type class-name :: operator op( arg-list)
{
function body
}
```

Where return type is the type of value returned by the specified operation and op is the operator being overloaded. The op is preceded by the keyword operator; operator op is the function name.

operator functions must be either member function, or friend function. A basic difference between them is that a friend function will have only one argument for unary operators and two for binary operators whereas member function will have only no argument for unary operators and one for binary operators, This is because the object used to invoke the member function is passed implicitly and therefore is available for the member functions. Arguments may be either by value or by reference.

operator functions are declared in the class using prototypes as follows:-

```
vector operator + (vector); /./ vector addition
vector operator-( ); //unary minus
friend vector operator + (vector, vector); // vector add
friend vector operator -(vector); // unary minus
vector operator - ( vector &a); // substraction
int operator = =(vector); //comparision
friend int operator = =(vector , vector); // comparision
```

vector is a data type of class and may represent both magnitude and direction or a series of points called elements.

The process of overloading involves the following steps:-

1. Create a class that defines the data type that is used in the overloading operation.

2. Declare the operator function operator op() in the public part of the class

3. It may be either a member function or friend function.

4. Define the operator function to implement the required operations.

GATEWAY CLASSES

Overloaded operator functions can be invoked by expressions such as
op x or x op;
for unary operators and
x op y
for binary opearators.
operator op(x);
for unary operator using friend function
operator op(x,y);
for binary operator using friend function.

We can overload all the C++ operators except the following:
- Class members access operator (. , .*)
- Scope resolution operator (: :)
- Size operator(sizeof)
- Condition operator (? :)

Unary -- operator overloading (using member function):

```
#include <iostream>
using namespace std;

class abc
{
	int m,n;
	public:
	abc()
	{
		m=8;
		n=9;
	}
	void show()
	{
		cout<<"m= "<<m<<endl;
		cout<<"n= "<<n<<endl;
	}
	operator -- ()
	{
		--m;
		--n;
	}
};
int main()
{
	abc x;
	cout<<"Before decrement: "<<endl;
	x.show();
	--x;
	cout<<"After decrement: "<<endl;
```

GATEWAY CLASSES

```
	x.show();
	return 0;
}
```

Output:

```
Before decrement:
m= 8
n= 9
After decrement:
m= 7
n= 8
```

Unary -- operator overloading (using friend function):

```
#include <iostream>
using namespace std;

class abc
{
	int m,n;
	public:
	abc()
	{
		m=8;
		n=9;
	}
	void show()
	{
		cout<<"m= "<<m<<endl;
		cout<<"n= "<<n<<endl;
	}
	friend operator--(abc& p);
};
operator -- (abc &p)
{
	--p.m;
	--p.n;
}
int main()
{
	abc x;
	cout<<"Before decrement: "<<endl;
	x.show();
	--x;
	cout<<"After decrement: "<<endl;
	x.show();
```

GATEWAY CLASSES

```
	return 0;
}
```

Output:

Before decrement:
m= 8
n= 9
After decrement:
m= 7
n= 8

Binary operator+ for adding two complex numbers (using member function)

```
#include <iostream>
using namespace std;

class complex
{
	float real,img;
	public:
		complex()
		{
			real=0;
			img=0;
		}
		complex(float r,float i)
		{
			real=r;
			img=i;
		}
		void show()
		{
			cout<<real<<"+i"<<img<<endl;
		}
		complex operator+(complex &p)
		{
			complex w;
			w.real=real+p.real;
			w.img=img+p.img;
			return w;
		}
};
int main()
{
	complex s(3,4);
	complex t(4,5);
	complex m;
```

GATEWAY CLASSES

```
	m=s+t;
	s.show();
	t.show();
	m.show();

	return 0;
}
```

Output:

```
3+i4
4+i5
7+i9
```

Binary operator+ for adding two complex numbers (using friend function)

```
#include <iostream>
using namespace std;

class complex
{
	float real,img;
	public:
	complex()
	{
		real=0;
		img=0;
	}
	complex(float r,float i)
	{
		real=r;
		img=i;
	}
	void show()
	{
		cout<<real<<"+i"<<img<<endl;
	}
	friend complex operator+(complex &p,complex &q);
};
complex operator+(complex &p,complex &q)
{
	complex w;
	w.real=p.real+q.real;
	w.img=p.img+q.img;
	return w;
}

int main()
{
```

GATEWAY CLASSES

```
        complex s(3,4);
        complex t(4,5);
        complex m;
        //m=operator+(s,t); OR
        m=s+t;
        s.show();
        t.show();
        m.show();

        return 0;
}
```

Output:

```
3+i4
4+i5
7+i9
```

GATEWAY CLASSES

Inheritance

C++ strongly supports the concept of reusability. The C++ classes can be used again in several ways. Once a class has been written and tested, it can be adopted by other programmers. This is basically created by defining the new classes, reusing the properties of existing ones. The mechanism of deriving a new class from an old one is called 'INHERTTENCE'. This is often referred to as IS-A' relationship because very object of the class being defined "is" also an object of inherited class. The old class is called 'BASE' class and the new one is called 'DERIVED' class.

Defining Derived Classes

A derived class is specified by defining its relationship with the base class in addition to its own details. The general syntax of defining a derived class is as follows:

```
class d_classname : Access specifier base_classname
{
__
__ // members of derived class
};
```

The colon indicates that the d_classname is derived from the base_classname. The access specifier or the visibility mode is optional and, if present, may be public, private or protected. By default it is private. Visibility mode describes the status of derived features e.g.

```
class xyz //base class
{
        members of xyz
};
class ABC : public xyz //public derivation
{
        members of ABC
};
class ABC : XYZ //private derivation (by default)
{
        members of ABC
};
```

GATEWAY CLASSES

In the inheritance, some of the base class data elements and member functions are inherited into the derived class. We can add our own data and member functions and thus extend the functionality of the base class. Inheritance, when used to modify and extend the capabilities of the existing classes, becomes a very powerful tool for incremental program development.

Single Inheritance

When a class inherits from a single base class, it is known as single inheritance.

Following program shows the single inheritance using public derivation.

```
#include <iostream>
using namespace std;

class worker
{
        int age;
        char name[10];
        public:
                void get();
                void show();
```

```
};
void worker::get()
{
	cout <<"Your name please: ";
	cin >> name;
	cout <<"Your age please: ";
	cin >> age;
}
void worker::show()
{
	cout <<"\nMy name is "<<name<<" and My age is "<<age<<endl;
}
class manager:public worker //derived class (publicly)
{
	int now;
	public:
		void get();
		void show();
};
void manager::get()
{
	worker::get() ; //the calling of base class input function.
	cout << "Number of workers under you: ";
	cin >> now;
}
void manager::show()
{
	worker::show ( ); //calling of base class output function.
	cout <<"No. of workers under me are " <<now<<endl;
}
int	main()
{
	worker W1;
	manager M1;
	M1.get();
	M1.show();

	return 0;
}
```

Output:

Your name please: Sohan
Your age please: 23
Number of workers under you: 34

My name is Sohan and My age is 23
No. of workers under me are 34

The following program shows the single inheritance by private derivation.

```
#include <iostream>
using namespace std;

class worker
{
	int age;
	char name[10];
	public:
```

GATEWAY CLASSES

```
                void get();
                void show();
};
void worker::get()
{
        cout <<"Your name please: ";
        cin >> name;
        cout <<"Your age please: ";
        cin >> age;
}
void worker::show()
{
        cout <<"\nMy name is "<<name<<" and My age is "<<age<<endl;
}
class manager: worker //derived class (privately by default)
{
        int now;
        public:
                void get();
                void show();
};
void manager::get()
{
        worker::get() ; //the calling of base class input function.
        cout << "Number of workers under you: ";
        cin >> now;
}
void manager::show()
{
        worker::show ( ); //calling of base class output function.
        cout <<"No. of workers under me are " <<now<<endl;
}
int     main()
{
        worker W1;
        manager M1;
        M1.get();
        M1.show();

        return 0;
}
```

GATEWAY CLASSES

Visibility Modes

Basically we have visibility modes to specify that in which mode you are deriving the another class from the already existing base class. They are:

a. **Private:** when a base class is privately inherited by a derived class, 'public members' of the base class become private members of the derived class and therefore the public members of the base class can be accessed by its own objects using the dot operator. The result is that we have no member of base class that is accessible to the objects of the derived class.

b. **Public:** On the other hand, when the base class is publicly inherited, 'public members' of the base class become 'public members' of derived class and therefore they are accessible to the objects of the derived class.

c. **Protected:** C++ provides a third visibility modifier, protected, which serve a little purpose in the inheritance. A member declared as protected is accessible by the member functions within its class and any class immediately derived from it. It cannot be accessed by functions outside these two classes.

The below mentioned table summarizes how the visibility of members undergo modifications when they are inherited

Base Class Visibility	Derived Class Visibility		
	Public	Private	Protected
Private	X	X	X
Public	Public	Private	Protected
Protected	Protected	Private	Protected

The protected members of a class can be accessed by:
a. A function i.e. friend of a class.
b. A member function of a class that is the friend of the class.
c. A member function of a derived class.

The following program shows the single inheritance using protected derivation

```
#include <iostream>
using namespace std;

class worker
{
	int age;
	char name[10];
	public:
		void get();
		void show();
};
void worker::get()
{
	cout <<"Your name please: ";
	cin >> name;
	cout <<"Your age please: ";
	cin >> age;
}
void worker::show()
{
	cout <<"\nMy name is "<<name<<" and My age is "<<age<<endl;
}
class manager: protected worker //derived class (protected)
{
	int now;
	public:
		void get();
		void show();
};
void manager::get()
{
	worker::get() ; //the calling of base class input function.
	cout << "Number of workers under you: ";
	cin >> now;
```

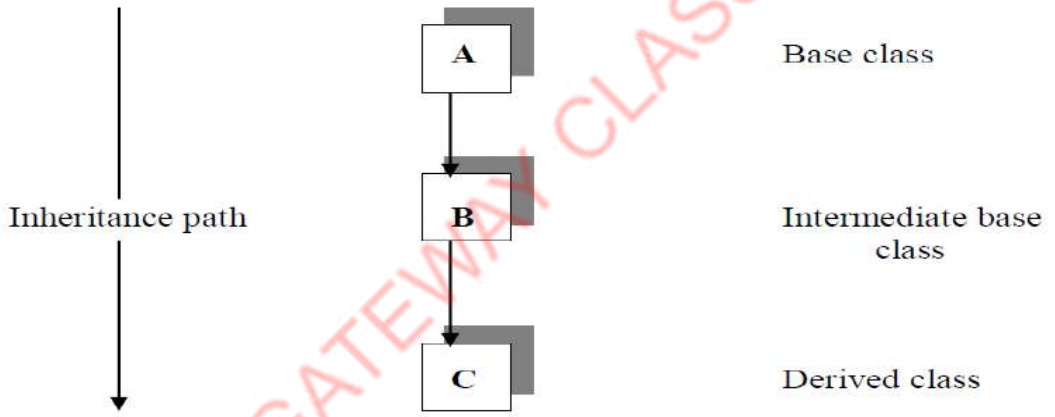
GATEWAY CLASSES

```
}
void manager::show()
{
        worker::show ( ); //calling of base class output function.
        cout <<"No. of workers under me are " <<now<<endl;
}
int main()
{
        worker W1;
        manager M1;
        M1.get();
        M1.show();

        return 0;
}
```

Multilevel Inheritance

When the inheritance is such that, the class A serves as a base class for a derived class B which in turn serves as a base class for the derived class C. This type of inheritance is called 'MULTILEVEL INHERITENCE'. The class B is known as the 'INTERMEDIATE BASE CLASS' since it provides a link for the inheritance between A and C. The chain ABC is called 'ITNHERITENCE PATH'.



GATEWAY CLASSES

The declaration for the same would be:

```
class A
{
//body
}
class B : public A
{
//body
}
class C : public B
{
//body
}
```

This declaration will form the different levels of inheritance.

Following program exhibits the multilevel inheritance.

```
#include <iostream>
using namespace std;
```

```
class worker
{
        int age;
        char name[10];
        public:
                void get();
                void show();
};
void worker::get()
{
        cout <<"Your name please: ";
        cin >> name;
        cout <<"Your age please: ";
        cin >> age;
}
void worker::show()
{
        cout <<"\nMy name is "<<name<<" and My age is "<<age<<endl;
}
class manager: public worker //derived class (protected)
{
        int now;
        public:
                void get();
                void show();
};
void manager::get()
{
        worker::get() ; //the calling of base class input function.
        cout << "Number of workers under you: ";
        cin >> now;
}
void manager::show()
{
        worker::show ( ); //calling of base class output function.
        cout <<"No. of workers under me are " <<now<<endl;
}
class ceo: public manager //declaration of derived class
{ //publicly inherited from the
        int nom; //intermediate base class
        public:
                void get ( ) ;
                void show ( ) ;
};
void ceo :: get ( )
{
        manager :: get ( ) ;
        cout << "No. of managers under you are: ";
        cin >> nom;
}
void ceo :: show ( )
{
        manager :: show ( ) ;
        cout << "The no. of managers under me are: ";
        cout << nom;
}

int     main()
```

GETEWAY CLASSES

```
{
        ceo cl ;
        cl.get ( ) ;
        cout << "\n";
        cl.show ( ) ;


        return 0;
}
```

Output:

```
Your name please: Sohan
Your age please: 25
Number of workers under you: 4
No. of managers under you are: 2

My name is Sohan and My age is 25
No. of workers under me are 4
The no. of managers under me are: 2
```

Multiple Inheritance

A class can inherit the attributes and behavior of two or more classes. This mechanism is known as 'MULTIPLE INHERITENCE'. Multiple inheritance allows us to combine the features of several existing classes as a starring point for defining new classes. It is like the child inheriting the physical feature of one parent and the intelligence of another. The syntax of the derived class is as follows:

```
class derived : visibility basel, visibility base2
{
        //body3
}
```

Where the visibility refers to the access specifiers i.e. public, private or protected. Following program shows the multiple inheritance.

GATEWAY CLASSES

```
#include <iostream>
using namespace std;

class father //Declaration of base classl
{
        int age ;
        char name [20] ;
        public:
                void get ( ) ;
                void show ( ) ;
};
void father :: get ( )
{
        cout << "your father name please";
        cin >> name;
        cout << "Enter the age";
        cin >> age;
}
void father :: show ( )
{
        cout<< "In my father's name is: "<<name<< "and my father's age is: "<<age;
}
class mother //Declaration of base class 2
```

```
{
        char name [20] ;
        int age ;
        public:
                void get ( )
                {
                        cout << "mother's name please" << "In";
                        cin >> name;
                        cout << "mother's age please" << "in";
                        cin >> age;
                }
                void show ( )
                {
                        cout << "In my mother's name is: " <<name;
                        cout << "In my mother's age is: " <<age;
                }
};
class daughter : public father, public mother //derived class inheriting
{ //publicly
        char name [20] ; //the features of both the base class
        int std;
        public:
                void get ( ) ;
                void show ( ) ;
};
void daughter :: get ( )
{
        father :: get ( ) ;
        mother :: get ( ) ;
        cout << "child's name: ";
        cin >> name;
        cout << "child's standard: ";
        cin >> std;
}
void daughter :: show ( )
{
        father :: show ( );
        mother :: show ( ) ;
        cout << "\n child's name is: " <<name;
        cout << "\n child's standard: " << std;
}
int main ( )
{
        daughter d1;
        d1.get ( ) ;
        d1.show ( ) ;

        return 0;
}
```

Output:

```
your father name pleaseSohan
Enter the age45
mother's name pleaseInMeera
mother's age pleasein40
child's name: Smriti
child's standard: 9
In my father's name is: Sohanand my father's age is: 45In my mother's name is: M
eeraIn my mother's age is: 40
```

GATEWAY CLASSES

```
child's name is: Smriti
child's standard: 9
```

Hierarchical Inheritance

Another interesting application of inheritance is to use is as a support to a hierarchical design of a class program. Many programming problems can be cast into a hierarchy where certain features of one level are shared by many others below that level.

In general the syntax is given as

```
class A
{
        // body A
}
class B: public A
{
        //body B
}
class C: public A
{
        //body C
}
```

In C++, such problems can be easily converted into hierarchies. The base class will include all the features that are common to the subclasses. A sub-class can be constructed by inheriting the features of base class and so on.

```
#include <iostream>
using namespace std;

class father //Declaration of base classl
{
        int age ;
        char name [20] ;
        public:
                void get ( ) ;
                void show ( ) ;
};
void father :: get ( )
{
        cout << "your father name please";
        cin >> name;
        cout << "Enter the age";
        cin >> age;
}
void father :: show ( )
{
        cout<< "\n My father's name is: "<<name<< " and my father's age is: "<<age;
}

class daughter : public father //derived class inheriting
{ //publicly
        char name [20] ; //the features of both the base class
        int std;
        public:
```

GATEWAY CLASSES

```
                void get ( ) ;
                void show ( ) ;
};
void daughter :: get ( )
{
        father :: get ( ) ;
                cout << "child's name: ";
        cin >> name;
        cout << "child's standard: ";
        cin >> std;
}
void daughter :: show ( )
{
        father :: show ( );
        cout << "\n child's name is: " <<name;
        cout << "\n child's standard: " << std;
}

class son : public father //derived class inheriting
{ //publicly
        char name [20] ; //the features of both the base class
        int std;
        public:
                void get ( ) ;
                void show ( ) ;
};
void son :: get ( )
{
        father :: get ( ) ;
        cout << "child's name: ";
        cin >> name;
        cout << "child's standard: ";
        cin >> std;
}
void son :: show ( )
{
        father :: show ( );
        cout << "\n child's name is: " <<name;
        cout << "\n child's standard: " << std;
}

int main ( )
{
        daughter d1;
        d1.get ( ) ;
        d1.show ( ) ;

        son s1;
        s1.get ( ) ;
        s1.show ( ) ;

        return 0;
}
```

Output:

```
your father name pleaseSohan
Enter the age46
```

GATEWAY CLASSES

child's name: Raina
child's standard: 8

My father's name is: Sohan and my father's age is: 46
child's name is: Raina
child's standard: 8your father name pleaseSohan
Enter the age46
child's name: Kushagra
child's standard: 6

My father's name is: Sohan and my father's age is: 46
child's name is: Kushagra
child's standard: 6

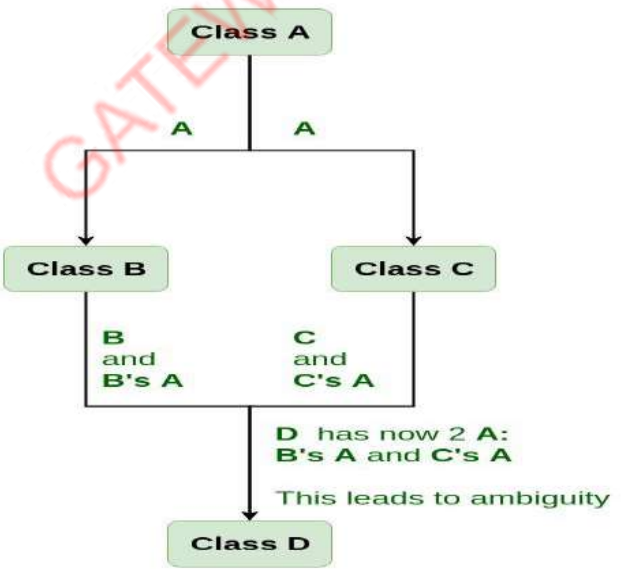
Hybrid Inheritance

There could be situations where we need to apply two or more types of inheritance to design a program. Basically Hybrid Inheritance is the combination of one or more types of the inheritance.

Virtual Base Classes

Consider a situation, where all the three kinds of inheritance, namely multi-level, multiple and hierarchical are involved.
Let us say the 'child' has two direct base classes ‘parent1’ and ‘parent2’ which themselves has a common base class ‘grandparent’. The child inherits the traits of ‘grandparent’ via two separate paths. It can also be inherit directly as shown by the broken line. The grandparent is sometimes referred to as ‘INDIRECT BASE CLASS’. Now, the inheritance by the child might cause some problems. All the public and protected members of ‘grandparent’ are inherited into ‘child’ twice, first via ‘parent1’ and again via ‘parent2’. So, there occurs a duplicacy which should be avoided.

GATEWAY CLASSES



The duplication of the inherited members can be avoided by making common base class as the virtual base class:

```
class g_parent
{
//Body
};
class parent1: virtual public g_parent
{
// Body
```

```
};
class parent2: public virtual g_parent
{
// Body
};
class child : public parent1, public parent2
{
// body
};
```

When a class is virtual base class, C++ takes necessary care to see that only one copy of that class is inherited, regardless of how many inheritance paths exists between virtual base class and derived class. Note that keywords ‘virtual’ and ‘public’ can be used in either order.

```
//Program to show hybrid inheritance using virtual base classes

#include<iostream>
using namespace std;

//Hybrid Inheritance
class Base
{
	public:
		int base;
		void setBase(int param)
		{
			base=param;
		}
		void getBase()
		{
			cout<<"base= "<<base<<endl;
		}
};

class A:public virtual Base//Hierarchical Inheritance
{
	protected:
		int a;
	public:
		void seta(int param)
		{
			a=param;
		}
		int get()
		{
			cout<<"a= "<<a<<endl;
		}
};
class B:virtual public Base//Hierarchical Inheritance
{
	protected:
		int b;
	public:
		void setb(int param)
		{
			b=param;
		}
		int get()
```

GATEWAY CLASSES

```
		{
			cout<<"b= "<<b<<endl;
		}
};
class C:public A,public B //MULTIPLE INHERITANCE
{
	int c;
	public:
		void setabc(int param1,int param2,int param3,int param4)
		{
			a=param1;
			b=param2;
			c=param3;
			base=param4;
		}
		void getabc()
		{
			A::get();
			B::get();
			cout<<"c= "<<c<<endl;
			cout<<"base= "<<base<<endl;
		}
};

int main()
{
	C obj;
	obj.setabc(4,5,6,7);
	obj.getabc();
	return 0;
}
```

GATEWAY CLASSES

Pure Virtual Functions and Abstract Classes

A ***pure virtual function*** (or abstract function) in C++ is a virtual function for which we don't have implementation, we only declare it. A pure virtual function is declared by assigning 0 in declaration.

An ***abstract class*** is a class in C++ which has at least one pure virtual function. Sometimes implementation of all function cannot be provided in a base class because we don't know the implementation. Such a class is called abstract class. For example, let Shape be a base class. We cannot provide implementation of function draw() in Shape, but we know every derived class must have implementation of draw(). Similarly an Animal class doesn't have implementation of move() (assuming that all animals move), but all animals must know how to move.

We cannot create objects of abstract classes.

```
// An abstract class
class Test
{
    // Data members of class
public:
    // Pure Virtual Function
    virtual void show() = 0;

   /* Other members */
};
```

GATEWAY CLASSES

- Abstract class can have normal functions and variables along with a pure virtual function.
- An abstract class can also have constructors.
- Abstract class cannot be instantiated, but pointers and references of Abstract class type can be created.
- If an Abstract Class has derived class, they must implement all pure virtual functions, or else they will become Abstract too.
- We can't create object of abstract class as we reserve a slot for a pure virtual function in Vtable, but we don't put any address, so Vtable will remain incomplete.

A pure virtual function is implemented by classes which are derived from a Abstract class. Following is a simple example to demonstrate the same.

```
#include<iostream>
using namespace std;

class Base
{
   int x;
public:
    virtual void fun() = 0;
```

```
    int getX() { return x; }
};

// This class inherits from Base and implements fun()
class Derived: public Base
{
    int y;
public:
    void fun() { cout << "fun() called"; }
};

int main(void)
{
    Derived d;
    d.fun();
    return 0;
}
```

Output:

```
fun() called
```

GATEWAY CLASSES

Virtual Functions

A virtual function is a member function which is declared within a base class and is re-defined (overridden) by a derived class. When you refer to a derived class object using a pointer or a reference to the base class, you can call a virtual function for that object and execute the derived class's version of the function.

- Virtual functions ensure that the correct function is called for an object, regardless of the type of reference (or pointer) used for function call.
- They are mainly used to achieve Runtime polymorphism
- Functions are declared with a **virtual** keyword in base class.
- The resolving of function call is done at run-time.

Rules for Virtual Functions

1. Virtual functions cannot be static and also cannot be a friend function of another class.
2. Virtual functions should be accessed using pointer or reference of base class type to achieve run time polymorphism.
3. The prototype of virtual functions should be same in base as well as derived class.
4. They are always defined in base class and overridden in derived class. It is not mandatory for derived class to override (or re-define the virtual function), in that case base class version of function is used.
5. A class may have virtual destructor but it cannot have a virtual constructor.

Compile-time (early binding) VS run-time (late binding) behavior of Virtual Functions

Consider the following simple program showing run-time behavior of virtual functions.

```
#include <iostream>
using namespace std;

class base {
public:
    virtual void print()
    {
        cout << "print base class" << endl;
    }

    void show()
    {
        cout << "show base class" << endl;
    }
};

class derived : public base {
public:
    void print()
    {
        cout << "print derived class" << endl;
    }
```

GATEWAY CLASSES

```
    void show()
    {
        cout << "show derived class" << endl;
    }
};

int main()
{
    base* bptr;
    derived d;
    bptr = &d;

    // Virtual function, bound at runtime - Late Binding
    bptr->print();

    // Non-virtual function, bound at compile time - Early Binding
    bptr->show();
}
```

Output:

```
print derived class
show base class
```

Runtime polymorphism is achieved only through a pointer (or reference) of base class type. Also, a base class pointer can point to the objects of base class as well as to the objects of derived class. In above code, base class pointer 'bptr' contains the address of object 'd' of derived class.

Late binding (Runtime) is done in accordance with the content of pointer (i.e. location pointed to by pointer) and ***Early binding (Compile time)*** is done according to the type of pointer, since print() function is declared with virtual keyword so it will be bound at run-time (output is *print derived class* as pointer is pointing to object of derived class) and show() is non-virtual so it will be bound during compile time (output is *show base class* as pointer is of base type).

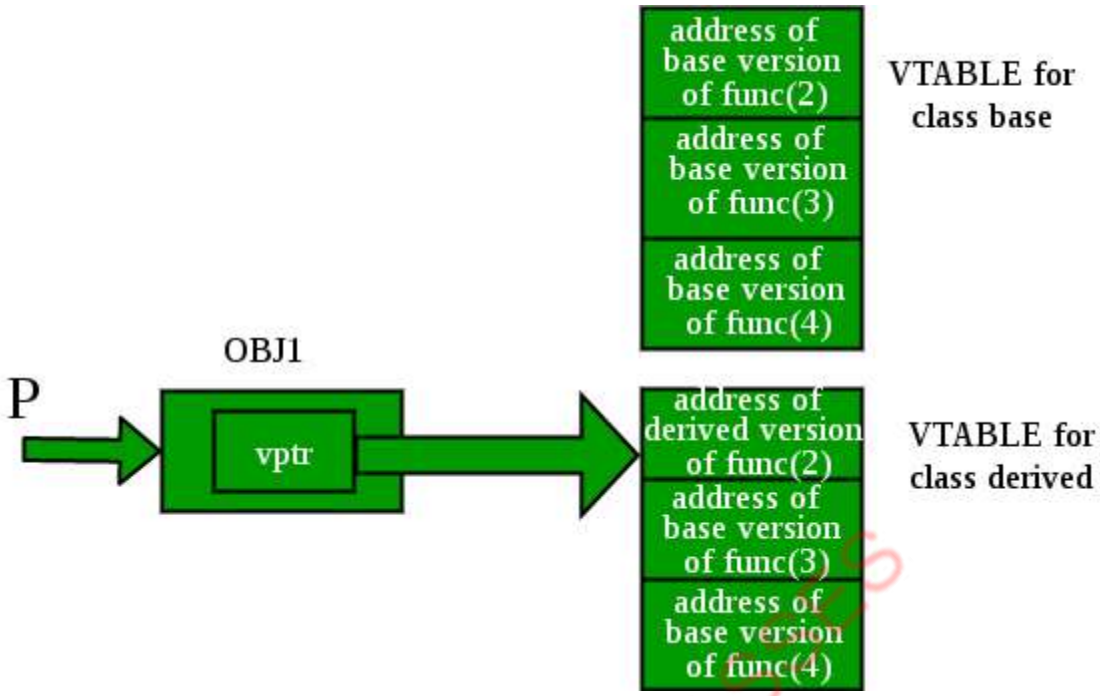
GATEWAY CLASSES

Working of Virtual Functions (VTABLE and VPTR)

If a class contains a virtual function then compiler itself does two things:

1. If object of that class is created then a **virtual pointer(VPTR)** is inserted as a data member of the class to point to VTABLE of that class. For each new object created, a new virtual pointer is inserted as a data member of that class.
2. Irrespective of object is created or not, **a static array of function pointer called VTABLE** is created where each cell contains the address of each virtual function contained in that class.

Consider the example below:



GATEWAY CLASSES

```
#include <iostream>
using namespace std;

class base {
public:
    void fun_1() { cout << "base-1\n"; }
    virtual void fun_2() { cout << "base-2\n"; }
    virtual void fun_3() { cout << "base-3\n"; }
    virtual void fun_4() { cout << "base-4\n"; }
};

class derived : public base {
public:
    void fun_1() { cout << "derived-1\n"; }
    void fun_2() { cout << "derived-2\n"; }
    void fun_4(int x) { cout << "derived-4\n"; }
};

int main()
{
    base* p;
    derived obj1;
    p = &obj1;

    // Early binding because fun1() is non-virtual
    // in base
    p->fun_1();

    // Late binding (RTP)
    p->fun_2();
```

```
    // Late binding (RTP)
    p->fun_3();

    // Late binding (RTP)
    p->fun_4();

    // Early binding but this function call is
    // illegal(produces error) becasue pointer
    // is of base type and function is of
    // derived class
    // p->fun_4(5);
}
```

Output:

```
base-1
derived-2
base-3
base-4
```

Initially, we create a pointer of type base class and initialize it with the address of the derived class object. When we create an object of the derived class, the compiler creates a pointer as a data member of the class containing the address of VTABLE of the derived class.

Similar concept of **Late and Early Binding** is used as in above example. For fun_1() function call, base class version of function is called, fun_2() is overridden in derived class so derived class version is called, fun_3() is not overridden in derived class and is virtual function so base class version is called, similarly fun_4() is not overridden so base class version is called. fun_4(int) in derived class is different from virtual function fun_4() in base class as prototypes of both the function are different.

GATEMAT CLASSES

Thank you

GATEWAY CLASSES